



UNIVERSIDADE FEDERAL DO AMAZONAS
PRÓ-REITORIA DE PESQUISA E PÓS-GRADUAÇÃO
DEPARTAMENTO DE APOIO À PESQUISA
PROGRAMA INSTITUCIONAL DE BOLSAS DE INICIAÇÃO CIENTÍFICA
FACULDADE DE TECNOLOGIA — ENGENHARIA DA COMPUTAÇÃO

RELATÓRIO FINAL
PIBIC 2025 — 2026

**VERIFICAÇÃO FORMAL DE SISTEMAS DE
INTELIGÊNCIA ARTIFICIAL GENERATIVA E DE
CONTROLADORES NEURAIS COM O *MODEL CHECKER*
ESBMC**

Bolsista: Matheus Serrão Uchôa
Orientador: Prof. Dr. Iury Valente de Bessa
Curso: Engenharia da Computação, Faculdade de Tecnologia
Área do CNPq: Ciência da Computação
Modalidade: **[preencher: PIBIC / PIBIC-AF / PAIC]**
Nº do processo: **[preencher: código do projeto, ex. PIB-E/0000/2025]**
Vigência: agosto de 2025 a julho de 2026
Repositório: diretório pibic/ do repositório do projeto

MANAUS — AMAZONAS
2026

RESUMO

Sistemas de inteligência artificial vêm sendo incorporados a malhas de controle e a cadeias de decisão em que uma saída fora de faixa deixa de ser um erro estatístico e passa a ser uma falha de segurança. Este relatório apresenta os resultados do plano de iniciação científica que investigou a aplicação de *bounded model checking* baseado em SMT, por meio do verificador ESBMC, a sete classes de artefatos de IA: redes neurais quantizadas em ponto fixo, *kernels* de inferência em C/C++, blocos *feed-forward* extraídos de modelos de linguagem (GPT-2 e LLaMA-7B), um ciclo neuro-simbólico de geração e correção de código, um controlador PID sob ruído não determinístico, uma política de aprendizado por reforço e, por fim, uma malha fechada completa formada por planta *cart-pole* e controlador DDPG quantizado. A contribuição metodológica central não está no número de propriedades provadas, mas na disciplina de evidência adotada: todo veredito relatado aqui é acompanhado da saída bruta do *solver*, do comando exato, das *flags*, do limite de tempo e do ambiente de execução, e *timeout* é tratado como resultado **indeciso**, nunca como prova de segurança. Sob esse critério, dos doze *harnesses* reexecutáveis do repositório, nove foram provados, três produziram contraexemplo e nenhum ficou indeciso. Duas propriedades da malha fechada foram diagnosticadas como **vácuas** — eram decididas sem que os 745 parâmetros do controlador participassem da prova — e foram reescritas de modo a depender efetivamente da rede. A limitação central do método foi medida, e não estimada: provar K passos da malha fechada custa 0,3 s em $K=1$ e 909 s em $K=4$, de modo que a barreira prática do BMC neste sistema está em quatro passos. Para contorná-la, o sistema acoplado foi exportado como sistema de transição e submetido a *model checking* indutivo (IC3/PDR): no modelo sintético, o PDR produziu prova ilimitada em 0,37 s onde o BMC levou 909 s para quatro passos; sobre o controlador real, porém, nenhum dos dois métodos decide a instância. Conclui-se que a verificação formal de componentes de IA é viável e informativa em escala modular, que a maior ameaça à validade desse tipo de trabalho é a propriedade vácuca, e que o próximo passo não é trocar de motor, mas mudar a representação do problema para o nível de palavra.

Palavras-chave: verificação formal; ESBMC; *bounded model checking*; SMT; redes neurais quantizadas; IC3/PDR; segurança de sistemas de controle.

ABSTRACT

Artificial intelligence components are increasingly embedded in control loops and decision pipelines where an out-of-range output is no longer a statistical error but a safety failure. This report presents the results of an undergraduate research project that investigated SMT-based bounded model checking, through the ESBMC verifier, applied to seven classes of AI artifacts: fixed-point quantized neural networks, C/C++ inference kernels, feed-forward blocks extracted from language models (GPT-2 and LLaMA-7B), a neuro-symbolic code generation and repair loop, a PID controller under non-deterministic sensor noise, a reinforcement learning policy, and a full closed loop comprising a cart-pole plant and a quantized DDPG controller. The main methodological contribution is not the number of properties proved but the evidence discipline adopted: every verdict reported here is backed by the raw solver output, the exact command, the flags, the time limit and the execution environment, and *timeout* is treated as an **undecided** outcome, never as proof of safety. Under this criterion, out of the twelve reproducible harnesses in the repository, nine were proved, three produced counterexamples and none remained undecided. Two closed-loop properties were diagnosed as **vacuous** — they were decided without the controller’s 745 parameters taking part in the proof — and were rewritten so as to genuinely depend on the network. The central limitation of the method was measured rather than estimated: proving K steps of the closed loop costs 0.3 s at $K=1$ and 909 s at $K=4$, so the practical BMC wall for this system lies at four steps. To work around it, the coupled system was exported as a transition system and submitted to inductive model checking (IC3/PDR): on the synthetic model, PDR produced an unbounded proof in 0.37 s where BMC took 909 s for four steps; on the real controller, however, neither method decides the instance. We conclude that formal verification of AI components is feasible and informative at a modular scale, that the greatest threat to validity in this line of work is the vacuous property, and that the next step is not to switch engines but to change the problem representation to the word level.

Keywords: formal verification; ESBMC; bounded model checking; SMT; quantized neural networks; IC3/PDR; control system safety.

SUMÁRIO

RESUMO	i
ABSTRACT	ii
1 Introdução	1
2 Objetivos	2
2.1 Objetivo geral	2
2.2 Objetivos específicos e grau de cumprimento	2
3 Fundamentação teórica	4
3.1 Verificação formal e <i>model checking</i>	4
3.2 <i>Bounded model checking</i> e o horizonte K	4
3.3 Satisfatibilidade booleana e módulo teorias	4
3.4 O verificador ESBMC	5
3.5 Redes neurais quantizadas e aritmética Q8.8	5
3.6 Verificação de redes neurais: trabalhos relacionados	6
3.7 <i>Model checking</i> indutivo: IC3/PDR	6
4 Metodologia	7
4.1 Visão geral	7
4.2 A camada <code>core_verify</code> e o status tri-estado	7
4.3 Protocolo de evidência	8
4.4 Teste de vacuidade	9
4.5 Formulação dos estudos de caso	9
4.5.1 Caso 1 — rede neural quantizada e <i>stubs</i> de inferência	9
4.5.2 Caso 2 — <i>kernels</i> de inferência em C/C++	10
4.5.3 Caso 3 — ciclo neuro-simbólico de geração e correção	10
4.5.4 Caso 4 — controlador PID sob ruído não determinístico	10
4.5.5 Caso 5 — blocos <i>feed-forward</i> de modelos de linguagem	11
4.5.6 Caso 6 — política de aprendizado por reforço	11
4.5.7 Caso 7 — malha fechada <i>cart-pole</i> com controlador DDPG	12
4.6 Integração contínua	12
5 Resultados e discussão	12
5.1 Consolidado das evidências	12
5.2 Caso 1 — redes quantizadas e <i>stubs</i> de inferência	13
5.3 Caso 2 — <i>kernels</i> de inferência e a barreira de escalabilidade	14

5.4	Caso 3 — ciclo neuro-simbólico de geração e correção	15
5.5	Caso 4 — PID sob ruído não determinístico	16
5.5.1	Biblioteca de controle de terceiros	18
5.6	Caso 5 — blocos <i>feed-forward</i> de modelos de linguagem	19
5.7	Caso 6 — política de aprendizado por reforço	20
5.8	Caso 7 — malha fechada com controlador DDPG quantizado	20
5.8.1	Fidelidade da quantização	20
5.8.2	Duas propriedades vácuas	21
5.8.3	Envelope de segurança e a barreira do BMC	21
5.8.4	<i>Model checking</i> indutivo como alternativa	22
5.9	Síntese comparativa	23
6	Limitações, ameaças à validade e dificuldades encontradas	24
6.1	Limitações do que foi provado	24
6.2	Ameaças à validade	25
6.2.1	Validade de construção: a propriedade vácuua	25
6.2.2	Validade interna: fidelidade das traduções	25
6.2.3	Validade externa: escala e generalização	26
6.2.4	Validade de conclusão: procedência dos números	26
6.3	Dificuldades encontradas ao longo do ciclo	26
7	Conclusões	27
7.1	Perspectivas de continuidade	28
8	Cronograma de atividades executadas	28
8.1	Produtos gerados	29
8.2	Participação em eventos e produção associada	30
	REFERÊNCIAS	31
	APÊNDICE A – REPRODUTIBILIDADE	33
	APÊNDICE B – ÍNDICE DE EVIDÊNCIAS	35

LISTA DE FIGURAS

1	Fluxo de verificação formal com o ESBMC, do código-fonte ao veredito do <i>solver</i>	7
2	Substituição da ativação transcendental: SiLU exata contra a aproximação linear tratável pelo <i>solver</i> . O veredito obtido vale para a aproximação, não para a função original.	11
3	Superfície de saída do neurônio e distribuição amostral. A figura descreve a amostra e não constitui prova sobre o domínio contínuo.	14
4	Tempo por iteração do ciclo gerar–verificar–corrigir, com decomposição entre atraso do gerador e tempo de verificação.	15
5	Máquina de estados do protocolo neuro-simbólico de geração e reparo.	16
6	Evolução da temperatura, do sinal de controle e do erro de rastreamento sob cinco perfis simulados de ruído de sensor.	17
7	Retrato de fase do controlador PID sob perturbação.	18
8	Distribuição das ações da política sobre a caixa de estados do <i>harness</i> : a região que excede o limite do atuador é atingível.	20

LISTA DE TABELAS

1	Objetivos específicos do plano de trabalho e grau de cumprimento.	3
2	Ambiente da execução consolidada de evidências.	9
3	Vereditos medidos dos doze <i>harnesses</i> reexecutáveis.	13
4	Escalabilidade da verificação do <i>kernel</i> GEMM em função da dimensão N	14
5	Verificação de blocos FFN com sobreaproximação intervalar da ativação (Método 2).	19
6	Segurança em dois passos em função do limite admitido de $ \dot{\theta} $	21
7	<i>Bounded model checking</i> contra <i>model checking</i> indutivo.	22
8	Síntese dos sete estudos de caso: o que foi estabelecido e sob quais condições.	24
9	Cronograma executado, por bimestre da vigência 2025–2026.	29
10	Produtos do ciclo e respectivos artefatos de verificação.	30
11	Veredito, comando e log bruto de cada <i>harness</i>	35
12	Medições complementares e sua procedência.	35

1 INTRODUÇÃO

A incorporação de modelos de aprendizado de máquina a sistemas embarcados e a malhas de controle deslocou uma classe inteira de defeitos para fora do alcance dos métodos tradicionais de teste. Enquanto um modelo preditivo isolado pode ser avaliado por acurácia sobre um conjunto de validação, um controlador neural embarcado em uma planta física não admite esse critério: o que importa não é o erro médio, mas se existe *alguma* combinação de entradas, dentro do envelope operacional declarado, capaz de levar o sistema a um estado inseguro. Essa pergunta é de natureza exaustiva, e amostragem — por mais densa que seja — não a responde.

A verificação formal ataca exatamente essa lacuna. No paradigma de *bounded model checking* (BMC) baseado em SMT, o programa é desenrolado até um limite K , traduzido em uma fórmula lógica e submetido a um *solver* que decide se existe atribuição de entradas capaz de violar a propriedade especificada. Quando existe, o *solver* devolve um contraexemplo concreto — um traço reproduzível que exhibe a violação (CORDEIRO; FISCHER; MARQUES-SILVA, 2012). Aplicada a redes neurais, a técnica vem sendo empregada tanto para provar robustez local quanto para exibir entradas adversariais concretas (HUANG *et al.*, 2017). Quando não existe, obtém-se uma garantia matemática, ainda que limitada ao horizonte K e às hipóteses declaradas.

Aplicar esse maquinário a componentes de inteligência artificial impõe três dificuldades específicas. A primeira é de **escala**: uma camada densa de dimensão d gera $O(d^2)$ multiplicações simbólicas, e um bloco de *transformer* de produção opera com $d = 4096$. A segunda é de **aritmética**: funções de ativação transcendentais como a sigmoide e a SiLU não possuem axiomatização nas teorias SMT usuais, e o *solver* as trata como funções não interpretadas, devolvendo UNKNOWN. A terceira, menos discutida na literatura e a que se mostrou mais perigosa neste trabalho, é de **especificação**: uma propriedade mal formulada pode ser decidida sem que o modelo sob verificação participe da prova. O resultado é uma prova verdadeira sobre um objeto que não é o objeto de interesse — uma *propriedade vácuua*.

Este relatório apresenta o trabalho desenvolvido no ciclo PIBIC 2025–2026 em torno do verificador ESBMC (GADELHA *et al.*, 2018), um *model checker* de código aberto, baseado em SMT, com suporte a C, C++ e ponto flutuante. O projeto percorreu sete estudos de caso de complexidade crescente, do neurônio quantizado isolado à malha fechada completa formada por planta e controlador de aprendizado por reforço, e produziu, além dos vereditos, uma infraestrutura de reprodutibilidade: um invólucro Python sobre o binário do verificador, uma suíte de regressão que impede a confusão entre erro de execução e propriedade violada, e um gerador de evidências que reexecuta cada *harness* citado no texto e versiona a saída bruta do *solver*.

A justificativa para essa ênfase em procedência de evidência não é retórica. Uma auditoria sistemática do próprio repositório, conduzida na fase final do ciclo, encontrou afirmações quantitativas que os artefatos não sustentavam: tempos de verificação que provinham de valores fixos em *scripts* de plotagem, um experimento de escalabilidade cujo parâmetro de dimensão

nunca chegava ao código compilado, um orquestrador de verificação com o portão de chamada ao *solver* comentado, e duas propriedades de segurança que decidiam sem tocar nos pesos da rede. Nenhum desses defeitos produzia erro visível; todos produziam números plausíveis. A correção deles, documentada ao longo da Seção 5, é parte substantiva do resultado deste PIBIC.

O relatório está organizado da seguinte forma. A Seção 2 recupera os objetivos do plano de trabalho e indica o grau de cumprimento de cada um. A Seção 3 apresenta os fundamentos de *model checking* limitado e indutivo, de codificação SMT e de quantização em ponto fixo. A Seção 4 descreve a infraestrutura construída, o protocolo de evidência e a formulação de cada estudo de caso. A Seção 5 reporta e discute os resultados medidos. A Seção 6 explicita limitações, ameaças à validade e dificuldades enfrentadas. A Seção 7 sintetiza as conclusões e a Seção 8 registra o cronograma efetivamente executado.

2 OBJETIVOS

2.1 Objetivo geral

Investigar e estender metodologias de verificação formal baseadas em SMT, por meio do verificador ESBMC, para componentes de inteligência artificial generativa e de redes neurais quantizadas, de modo a obter garantias de segurança, limites operacionais estritos e correteude lógica tanto em abstrações de alto nível quanto em *kernels* de baixo nível.

2.2 Objetivos específicos e grau de cumprimento

O plano de trabalho previa cinco frentes, organizadas por complexidade arquitetural crescente. A Tabela 1 apresenta cada uma delas ao lado do que foi efetivamente entregue, indicando explicitamente onde o resultado ficou aquém do previsto e por quê. O detalhamento está na Seção 5.

Tabela 1: Objetivos específicos do plano de trabalho e grau de cumprimento.

Objetivo específico	Resultado obtido	Situação
Verificação de redes neurais quantizadas (simulação QNN)	Camada Q8.8 implementada e verificada em C; <i>harness</i> XOR provado em 0,08 s; o <i>frontend</i> Python do ESBMC não está disponível no binário utilizado	Cumprido com ressalva
Segurança de <i>kernels</i> e <i>stubs</i> de inferência	Contraexemplo de segurança de memória obtido em 0,43 s; GEMM com <i>tiling</i> provado até $N=3$; barreira de escalabilidade medida em $N=4$	Cumprido
Verificação de propriedades de sistema em malha fechada	Planta <i>cart-pole</i> e ator DDPG quantizado verificados acoplados; duas propriedades vácuas identificadas e reescritas; envelope de segurança caracterizado	Cumprido parcialmente
Arcabouço programático em Python e ciclo neuro-simbólico	API <i>core_verify</i> com status tri-estado e suíte de regressão; ciclo gerar-verificar-corrigir demonstrado com <i>mock</i> determinístico	Cumprido com ressalva
Garantias de segurança em aprendizado por reforço	Política de RL verificada; <i>solver</i> produziu contraexemplo: a ação ultrapassa o limite do atuador na caixa de estados analisada	Cumprido (resultado negativo)

Três observações são necessárias para que a tabela seja lida corretamente. Primeiro, “cumprido com ressalva” na primeira linha significa que o objetivo foi atingido em C, e não em Python: o binário do ESBMC versionado no repositório (6.8.0) não possui o *frontend* Python compilado e rejeita arquivos `.py`; concluir essa frente exige compilar a versão 8.0.0 com `ENABLE_PYTHON_FRONTEND=ON`. Segundo, na quarta linha, o ciclo neuro-simbólico foi demonstrado com um agente simulado determinístico, e não com um modelo de linguagem real — o experimento comprova o *protocolo* (gerar, verificar, extrair diagnóstico, corrigir), não a autonomia de um agente. Terceiro, o resultado negativo da última linha é reportado como resultado, e não como falha de execução: o *solver* exibiu um traço concreto em que a ação excede o intervalo $[-1, 1]$, de modo que o *safety shield* projetado ainda **não** está validado como bloqueio eficaz.

3 FUNDAMENTAÇÃO TEÓRICA

3.1 Verificação formal e *model checking*

Verificação formal designa o conjunto de técnicas que estabelecem, por meio matemático, a conformidade de um sistema a uma especificação. A distinção operacional em relação ao teste convencional é de cobertura: o teste amostra o espaço de execuções, ao passo que o *model checking* decide sobre a totalidade das execuções admitidas pelo modelo. Se a propriedade é violada, o procedimento devolve um contraexemplo, isto é, uma trajetória de execução que conduz ao estado de erro; se não é, devolve uma garantia relativa ao modelo e às hipóteses assumidas.

Essa última qualificação é essencial e será retomada ao longo do relatório: o veredito de um *model checker* vale para o modelo submetido, para as hipóteses declaradas e para o horizonte considerado. Transferi-lo automaticamente para o sistema implantado é um erro de inferência, não uma consequência da prova.

3.2 *Bounded model checking* e o horizonte K

No *bounded model checking*, os laços e as chamadas de função são desenrolados até um limite K , e a semântica do programa desenrolado é convertida em uma fórmula lógica $\varphi = I \wedge \bigwedge_{i=0}^{K-1} T(s_i, s_{i+1}) \wedge \neg P$, em que I descreve os estados iniciais, T a relação de transição e P a propriedade. A satisfatibilidade de φ equivale à existência de um contraexemplo de comprimento até K ; a insatisfatibilidade equivale à ausência de violação *dentro daquele horizonte*.

Duas consequências são diretas. A primeira: a fórmula contém K cópias da relação de transição, de modo que o custo cresce com K de forma que, na prática, se torna proibitiva bem antes do horizonte de interesse. A segunda: um resultado negativo é sempre condicional a K , e por isso “seguro até K passos” e “seguro” são afirmações distintas. A Seção 5 apresenta a medição direta dessa barreira no sistema estudado.

3.3 Satisfatibilidade booleana e módulo teorias

O problema SAT busca uma atribuição de valores-verdade que satisfaça uma fórmula proposicional. Sua eficiência prática é notável, mas sua expressividade é baixa para construções de linguagens de programação: inteiros com largura fixa, ponto flutuante, *arrays* e ponteiros teriam de ser codificados manualmente bit a bit.

SMT (*Satisfiability Modulo Theories*) estende o SAT acoplando ao motor booleano resolvidores especializados em teorias — vetores de bits, *arrays*, aritmética linear, ponto flutuante. Isso permite que o *model checker* traduza construções da linguagem para a teoria correspondente e delegue a decisão. Neste trabalho foram utilizados os *solvers* Z3 (MOURA; BJØRNER, 2008) e Boolector (NIEMETZ; PREINER; BIÈRE, 2014); o Bitwuzla (NIEMETZ; PREINER, 2023), sucessor do Boolector, está disponível apenas em versões mais recentes do

verificador.

A escolha do *solver* não é neutra em desempenho. Nos *harnesses* de aritmética inteira Q8.8 deste projeto, o Boolector foi consistentemente preferido por operar diretamente sobre a teoria de vetores de bits; o Z3 foi empregado nos *harnesses* que exigem `-floatbv`, isto é, semântica IEEE-754 explícita.

3.4 O verificador ESBMC

O ESBMC (GADELHA *et al.*, 2018) é um *model checker* limitado, baseado em SMT, com suporte a C, C++, Python e CUDA. Seu fluxo compreende quatro etapas: análise do código-fonte pelo *frontend*; conversão para um *GOTO-program*, isto é, um grafo de fluxo de controle simplificado; execução simbólica, que produz as equações de estado do programa desenrolado; e codificação dessas equações em uma fórmula SMT submetida ao *solver*.

Três primitivas estruturam a modelagem:

- `nondet_T()` introduz um valor não determinístico do tipo `T`, instruindo o *solver* a considerar todo o domínio daquele tipo;
- `__ESBMC_assume(e)` restringe o espaço de busca, descartando os caminhos em que `e` é falsa. Não é uma verificação: é uma hipótese. Toda conclusão obtida sob um `assume` é condicional a ele;
- `__ESBMC_assert(e, msg)` expressa a propriedade a provar. O verificador busca exaustivamente uma atribuição, compatível com os `assume` ativos, que torne `e` falsa.

```
float x = nondet_float();
__ESBMC_assume(x >= -10.0f && x <= 10.0f); // hipotese de ambiente
float y = model_predict(x);
__ESBMC_assert(y <= 1.0f, "saida excede o limite do atuador");
```

Listing 1: Padrão de modelagem empregado nos *harnesses*.

A relação entre `assume` e `assert` concentra o principal risco metodológico do método. Um `assume` excessivamente forte poda o espaço de busca a ponto de tornar a propriedade trivialmente verdadeira; no limite, um `assume` insatisfazível torna *qualquer* `assert` provável. Verificar que os `assume` de um *harness* são satisfazíveis — por exemplo, inserindo `assert(0)` logo após eles e exigindo que a prova **falhe** — é um teste de vacuidade barato e foi adotado como prática neste projeto.

3.5 Redes neurais quantizadas e aritmética Q8.8

Uma rede treinada em ponto flutuante de 32 bits é frequentemente inviável em hardware embarcado. A quantização converte pesos e ativações para inteiros de largura fixa: no formato Q8.8 empregado neste trabalho, um valor real v é representado pelo inteiro $\hat{v} = \text{round}(v \cdot 256)$, e

o produto de dois valores quantizados exige realinhamento por deslocamento à direita, uma vez que a escala é elevada ao quadrado na multiplicação.

Há duas razões para que a quantização favoreça a verificação. A primeira é semântica: a aritmética inteira é decidida na teoria de vetores de bits, muito mais tratável que a aritmética de ponto flutuante. A segunda é de fidelidade: o código verificado passa a ser aritmeticamente idêntico ao código executado no dispositivo, e não uma idealização em precisão infinita. O preço é que o truncamento introduz erro, e esse erro precisa ser medido, não presumido — a Seção 5 reporta a caracterização do erro de quantização do controlador estudado.

3.6 Verificação de redes neurais: trabalhos relacionados

A verificação de redes neurais consolidou-se a partir do Reluplex (KATZ *et al.*, 2017), que estendeu o algoritmo simplex para lidar com a não linearidade da ReLU, e do Marabou (KATZ *et al.*, 2019), sua evolução. O Neurify (WANG *et al.*, 2018) explorou a direção complementar, baseada em propagação de intervalos e refinamento simbólico. Esses trabalhos operam sobre a rede em precisão real.

A linha mais próxima deste projeto é a do QNNVerifier (SENA *et al.*, 2021; SONG *et al.*, 2022), que verifica a implementação *quantizada* da rede, considerando o comprimento finito de palavra do dispositivo alvo. A abordagem combina tradução do modelo para rotinas em C, reescrita das operações em ponto fixo, inferência estática de intervalos por análise de valores para podar o espaço de busca, e discretização por tabelas de consulta das funções de ativação transcendentais. Este trabalho reaproveita essa estrutura, com uma diferença de ênfase: aqui, os intervalos injetados por `assume` são tratados explicitamente como *sobreaproximação*, e a solidez do veredito é declarada como condicional ao conservadorismo desses limites.

3.7 Model checking indutivo: IC3/PDR

O IC3 (BRADLEY, 2011), na formulação PDR de Eén, Mishchenko e Brayton (2011), ataca a limitação estrutural do BMC. Em vez de construir uma fórmula com K cópias da relação de transição, o algoritmo mantém uma sequência de *frames* $F_0, F_1, \dots$ que sobreaproximam os estados alcançáveis em até i passos, e refina cada um deles por consultas que envolvem *uma única* cópia da transição. Quando dois *frames* consecutivos convergem, o resultado é um **invariante indutivo**: uma prova válida para todo tempo, não apenas até um horizonte.

A contrapartida é que o IC3 opera sobre sistemas de transição finitos, tipicamente circuitos sequenciais. Levar um programa a esse formato exige síntese: no fluxo adotado neste trabalho, o sistema acoplado é emitido como Verilog, sintetizado pelo Yosys (WOLF, 2016) e verificado pelo ABC (BRAYTON; MISHCHENKO, 2010). Essa tradução tem um custo conceitual relevante, discutido adiante: ela *bit-blasta* as palavras de estado, dissolvendo em bits isolados a estrutura sobre a qual o PDR generaliza cláusulas.

4 METODOLOGIA

4.1 Visão geral

A metodologia seguiu o ciclo canônico da verificação baseada em SMT, ilustrado na Figura 1: o artefato sob análise é reduzido a um *harness* em C ou C++ que declara as hipóteses de ambiente por `__ESBMC_assume` e a propriedade por `__ESBMC_assert`; o ESBMC converte esse *harness* em *GOTO-program*, executa-o simbolicamente e submete as condições de verificação ao *solver*, que devolve prova ou contraexemplo.

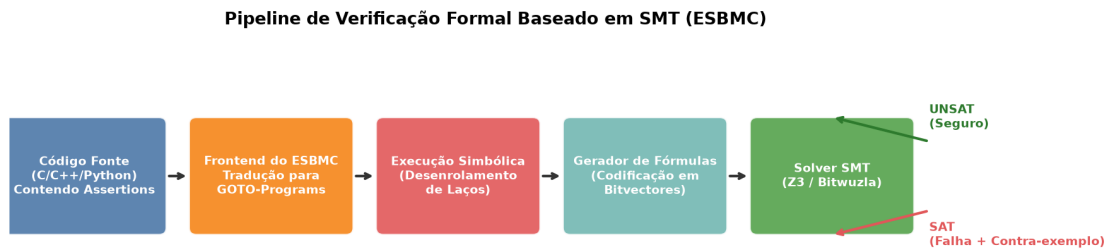


Figura 1: Fluxo de verificação formal com o ESBMC, do código-fonte ao veredito do *solver*.

Fonte: elaborada pelo autor.

Sobre esse ciclo foram construídas três camadas metodológicas que constituem a contribuição de infraestrutura do projeto: um invólucro programático que classifica corretamente o resultado do verificador (Seção 4.2), um protocolo de evidência que torna cada número reexecutável (Seção 4.3), e um teste de vacuidade que impede que uma propriedade seja aceita sem que o modelo participe da prova (Seção 4.4).

4.2 A camada `core_verify` e o status tri-estado

A automação da verificação exige interpretar a saída do ESBMC programaticamente. A primeira implementação do projeto reduzia essa interpretação a um booleano `is_safe`, derivado da presença da cadeia `VERIFICATION SUCCESSFUL` na saída padrão. Essa modelagem contém um defeito grave e silencioso: quando o verificador *não executa* — arquivo ausente, erro de compilação, *flag* inexistente, estouro de tempo — `is_safe` também é falso, e o código cliente que testa `not is_safe` conclui “bug encontrado” a partir de uma execução que nunca ocorreu.

O invólucro foi reescrito para derivar um status de seis valores, cruzando o código de retorno do processo com o marcador textual da saída:

```

from core_verify.esbmc_caller import run_esbmc, SAFE, UNSAFE

r = run_esbmc("harness.c", z3=True, floatbv=True, unwind=10)

```

```

r.status      # SAFE | UNSAFE | PARSE_ERROR | USAGE_ERROR | TIMEOUT |
              UNKNOWN
r.verificou   # True somente em SAFE ou UNSAFE

if r.status == UNSAFE:      # correto
    tratar_contraexemplo(r)
# if not r.is_safe:      # INCORRETO: verdadeiro tambem quando nada
# rodou

```

Listing 2: Interpretação correta do resultado da verificação.

Quando código de retorno e marcador discordam, o status resultante é UNKNOWN, e não um dos dois vereditos: a discordância indica que a execução não é confiável. Os códigos de retorno foram medidos diretamente sobre o binário 6.8.0 utilizado (0, 1, 6 e 64). O tratamento de *timeout* encerra o grupo de processos, e não apenas o processo filho, evitando que instâncias do *solver* sobrevivam à chamada.

Duas regressões foram acrescentadas à suíte para congelar decisões que já haviam sido tomadas erradas antes. A primeira garante que as *flags* `-bounds-check` e `-pointer-check`, que **não existem** no ESBMC — as verificações correspondentes são ativadas por padrão e a ferramenta expõe apenas as formas negativas — jamais voltem a ser emitidas; ao recebê-las, o verificador termina com código 64, que o invólucro classifica como `USAGE_ERROR` e não como propriedade violada. A segunda exercita os quatro modos de falha (arquivo ausente, fonte inválido, *flag* inválida e estouro de tempo) exigindo, em cada caso, que o resultado **não** seja confundido com UNSAFE.

4.3 Protocolo de evidência

Todo veredito citado neste relatório é produzido por um único ponto de entrada, `scripts/gerar_evidencias.py`, que executa cada *harness*, grava a saída **bruta** do *solver* em `results/logs/` e consolida a tabela afirmação → comando → veredito → log em `results/EVIDENCIAS.md`. O protocolo obedece a quatro regras:

1. a saída bruta é sempre preservada, inclusive quando o veredito é negativo ou indeciso, e nunca filtrada por palavra-chave antes de ser gravada;
2. *timeout* é registrado como **indeciso**, em coluna distinta de SAFE e de UNSAFE, e nunca colapsado com nenhum dos dois;
3. o ambiente de execução — versão do verificador, *solver*, *flags* completas, limite de tempo, sistema operacional e número de núcleos — acompanha cada medição;
4. falha de compilação aparece como `PARSE_ERROR`, e não como ausência de violação.

A Tabela 2 descreve o ambiente da execução consolidada reportada na Seção 5.

Tabela 2: Ambiente da execução consolidada de evidências.

Item	Valor
Verificador	ESBMC 6.8.0, 64 bits, x86_64-linux
<i>Solvers</i>	Boolector (padrão) e Z3 (<i>harnesses</i> com <code>-floatbv</code>)
Sistema operacional	Linux 6.18.5, x86_64, glibc 2.39
Processador	4 núcleos
<i>Harnesses</i>	12, todos reexecutáveis por <code>scripts/gerar_evidencias.py</code>
Duração total	463 s

Fonte: `results/EVIDENCIAS.md`, gerado por execução.

Registre-se que o binário versionado no repositório é a versão 6.8.0, enquanto a árvore de código do ESBMC presente no mesmo repositório é a 8.0.0, não compilada. Algumas *flags* existem apenas nesta última — entre elas `-std`, necessária para C++11, `-multi-property` e `-cvc4`. Essa diferença é a causa direta de dois bloqueios reportados na Seção 5.

4.4 Teste de vacuidade

Uma propriedade é **vácua** quando seu veredito não depende do artefato que se pretende verificar. O teste adotado é direto e barato: substitui-se o modelo por uma entrada livre e reexecuta-se a mesma asserção. Se o veredito e o contraexemplo se reproduzem inalterados, o modelo era código morto para aquela propriedade.

O procedimento é aplicado em duas direções complementares:

- **vacuidade do modelo:** a rede é removida e a propriedade reexecutada. Se ainda decide, ela não fala sobre a rede;
- **vacuidade das hipóteses:** insere-se `assert(0)` imediatamente após os `assume` do *harness*. Se essa asserção for *provada*, os `assume` são insatisfazíveis e todo veredito daquele *harness* é vazio.

Aplicado às propriedades da malha fechada, o teste revelou duas vacuidades, cuja correção é relatada na Seção 5.8. Aplicado às 24 hipóteses da segunda camada da rede de neurônios mortos, mostrou que nenhum `assume` é insatisfazível — o risco existia, mas não se materializou naquele conjunto.

4.5 Formulação dos estudos de caso

4.5.1 Caso 1 — rede neural quantizada e stubs de inferência

Uma perceptron multicamadas com ativação ReLU foi implementada em ponto fixo Q8.8, com as operações `fxp_add` e `fxp_mult` reproduzindo o truncamento do dispositivo alvo, e verificada sobre as quatro entradas discretas do problema XOR. Ao lado dela, foram verificados

um *stub* da mesma rede em ponto flutuante, um *harness* de propriedade sobre a rede completa e um *stub* de bloco MLP *gated* no estilo DeepSeek. A pré-condição de entrada é $x_1, x_2 \in [0, 1]$ e a propriedade de saída é um limite superior explícito sobre a predição desquantizada.

Neste caso foi reproduzida, em código, a estratégia de poda por intervalos do QNNVerifier: limites por neurônio são injetados como hipóteses, colapsando ramos que a propagação de valores mostra inalcançáveis. A operação é uma *sobreaproximação* e está declarada como tal — o veredito vale para a relaxação definida pelos intervalos.

4.5.2 Caso 2 — kernels de inferência em C/C++

Dois *kernels* análogos aos de motores de inferência foram verificados quanto a segurança de memória: um *kernel* de atenção por produto escalar com alocação dinâmica e comprimento de sequência simbólico `seq_len` $\in [1, 10]$, e um *kernel* de multiplicação de matrizes com *tiling*, parametrizado pela dimensão N para o estudo de escalabilidade. As verificações empregam `-memory-leak-check` e `-overflow-check`, com limite de tempo de 300 s.

4.5.3 Caso 3 — ciclo neuro-simbólico de geração e correção

O terceiro caso modela o protocolo em que um gerador produz código C, o ESBMC o verifica e, havendo contraexemplo, o diagnóstico volta ao gerador como entrada para correção. A iteração 0 introduz deliberadamente um *buffer overflow* por `strcpy` de 20 bytes em um destino de 10; a iteração 1 aplica a correção com `strncpy`.

O gerador é um **mock determinístico**, e não um modelo de linguagem. Essa escolha é uma limitação assumida do experimento e está registrada como tal: o que se demonstra é o protocolo e o custo do verificador dentro do ciclo, não a competência de um agente autônomo.

4.5.4 Caso 4 — controlador PID sob ruído não determinístico

Uma planta térmica de primeira ordem, com aquecimento proporcional à ação de controle e perda constante, é regulada por um PID digital com $K_p = 1,0$, $K_i = 0,1$, $K_d = 0,5$ e saturação em $[0, 100]$:

$$T_{k+1} = \max(20, T_k + (u_k r_h - r_c) \Delta t), \quad u_k = K_p e_k + K_i \sum_{j \leq k} e_j \Delta t + K_d \frac{e_k - e_{k-1}}{\Delta t}. \quad (1)$$

O ruído de sensor é escolhido pelo próprio *solver* a cada passo, restrito a $[-5, +5]^\circ\text{C}$, e a propriedade de segurança é a invariante $T_k < 150^\circ\text{C}$ para todos os dez passos. A verificação usa `-floatbv` e `-unwind 11`.

Complementarmente, avaliou-se a aplicabilidade da abordagem a uma biblioteca de controle real de terceiros: a *Arduino-PID-Library*, incorporada como submódulo Git fixado em *commit* determinado, com *harness* que instancia a classe `PID` sob a mesma planta e o mesmo modelo de ruído.

4.5.5 Caso 5 — blocos feed-forward de modelos de linguagem

Blocos FFN foram extraídos de modelos ONNX do GPT-2 e do LLaMA-7B e quantizados em Q8.8. Duas codificações foram comparadas:

- **Método 1 — proxy ReLU:** substitui GeLU/SiLU por ReLU. É mais rápido, mas verifica um modelo que não é o modelo implantado;
- **Método 2 — ativação abstrata:** substitui a ativação por hipóteses de intervalo por neurônio, no estilo do QNNVerifier. Produz uma sobreaproximação intervalar, conservadora *somente se* os limites cobrirem todas as saídas alcançáveis; a relação funcional exata não é codificada.

Os resultados reportados adiante correspondem ao Método 2. Os geradores de *harnesses* e as tabelas de consulta de GeLU e SiLU são reexecutáveis e produzem arquivos byte a byte idênticos aos versionados.

A dificuldade de fundo é que $\sigma(z) = 1/(1 + e^{-z})$ é transcendental e não possui axiomatização SMT, levando a UNKNOWN. Onde foi necessário manter a forma funcional, empregou-se a aproximação linear conservadora $\sigma_{\text{aprox}}(z) = \text{clip}(0,25z + 0,5, 0, 1)$, tangente de σ em $z = 0$ com saturação, ilustrada na Figura 2.

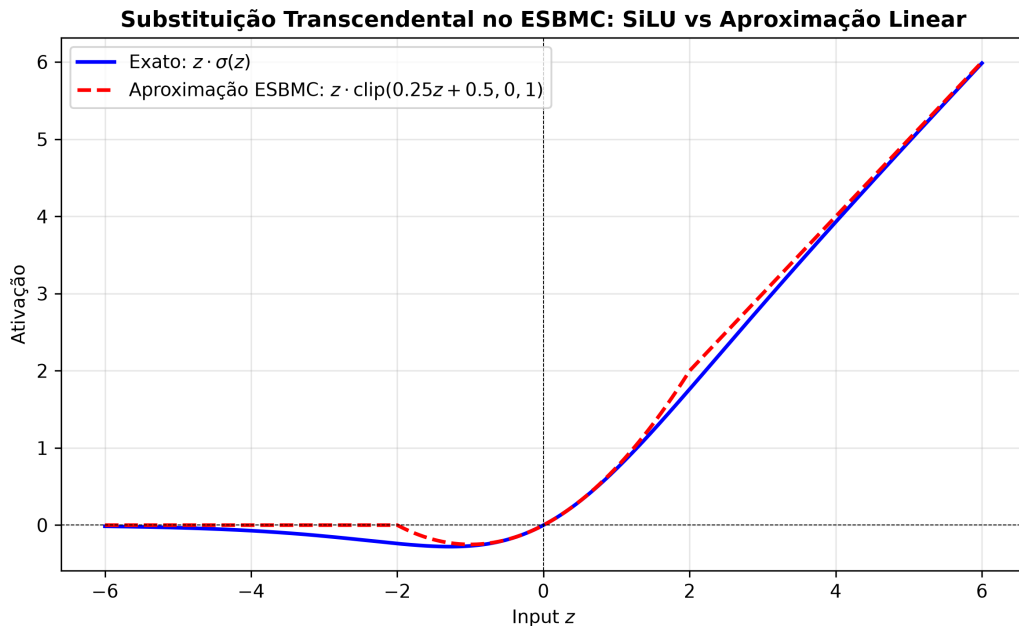


Figura 2: Substituição da ativação transcendental: SiLU exata contra a aproximação linear tratável pelo *solver*. O veredito obtido vale para a aproximação, não para a função original.

Fonte: elaborada pelo autor.

4.5.6 Caso 6 — política de aprendizado por reforço

Uma política de RL do tipo ator-crítico, com ação contínua em $[-1, 1]$, foi transcrita para C e submetida à busca não determinística sob `-floatbv`, com o objetivo de decidir se existe

estado, dentro da caixa admitida, para o qual a ação viola o limite do atuador.

4.5.7 Caso 7 — malha fechada cart-pole com controlador DDPG

Este é o único caso em que planta e controlador são verificados **acoplados**. O ator DDPG (LILLICRAP *et al.*, 2016), de arquitetura $4 \rightarrow 24 \rightarrow 24 \rightarrow 1$ e 745 parâmetros, foi quantizado em Q8.8 e integrado ao modelo do *cart-pole* (BARTO; SUTTON; ANDERSON, 1983). A aritmética de inferência está implementada três vezes — em Python para o treinamento, em TypeScript para a demonstração em navegador e em C para o *harness* — e as três coincidem bit a bit: mesma ordem de operações, mesmo truncamento em direção a zero e mesma aproximação de tanh em cinco ramos.

Para superar o horizonte do BMC, o sistema acoplado foi também exportado como sistema de transição: `gen_transition_system.py` lê os pesos reais e emite Verilog com largura configurável, o Yosys (WOLF, 2016) sintetiza o circuito e o ABC (BRAYTON; MISHCHENKO, 2010) executa `pdr` e `bmc3`. Antes de qualquer medição, `validate_forward.py` realiza um teste diferencial entre o Verilog gerado e a referência Q8.8 em doze estados amostrados; a coincidência detecta regressão de tradução, mas **não** constitui prova de equivalência universal entre as implementações.

4.6 Integração contínua

A suíte de verificação é executada a cada submissão por um fluxo de integração contínua. Duas decisões de projeto merecem registro por decorrerem de defeitos observados na primeira versão do fluxo. O binário do verificador é **versionado junto ao repositório** em vez de baixado a cada execução, o que elimina a divergência de versão entre ambiente local e remoto — fonte de resultados não reproduzíveis. E o caminho do binário é propagado por `$GITHUB_ENV`, e não por `export PATH`: variáveis exportadas em um passo não persistem para o seguinte, defeito que mantinha a etapa de verificação inoperante *sem emitir erro*.

Por decisão de escopo do ciclo, os resultados de execução remota em múltiplas plataformas não são apresentados como evidência neste relatório. A aceitação dos resultados apoia-se exclusivamente nas execuções locais documentadas na Seção 5; a estabilização em Windows, macOS e ARM permanece como trabalho separado.

5 RESULTADOS E DISCUSSÃO

5.1 Consolidado das evidências

A Tabela 3 apresenta o resultado da execução consolidada dos doze *harnesses* citados neste relatório. Cada linha corresponde a um arquivo de log bruto versionado, indexado no Apêndice B. O ambiente é o da Tabela 2.

Tabela 3: Vereditos medidos dos doze *harnesses* reexecutáveis.

Caso	<i>Harness</i>	Veredito	Tempo
Caso 1	MLP quantizada (XOR), Q8.8	SAFE	0,08 s
Caso 1	<i>Stub</i> da MLP em ponto flutuante	SAFE	3,09 s
Caso 1	Rede completa + propriedade de saída	UNSAFE	197,82 s
Caso 1	<i>Stub</i> de bloco MLP <i>gated</i>	SAFE	0,08 s
Caso 2	<i>Kernels</i> de inferência	UNSAFE	0,43 s
Caso 2	GEMM com <i>tiling</i> , $N = 3$	SAFE	17,51 s
Caso 4	PID, ruído $[-5, +5]$, 10 passos	SAFE	213,47 s
Caso 6	Política de RL, limite do atuador	UNSAFE	2,58 s
Caso 5	FFN GPT-2 2×4 , intervalo GeLU	SAFE	0,24 s
Caso 5	FFN LLaMA-7B 4×8 , intervalo SiLU	SAFE	3,46 s
Caso 5	FFN GPT-2 4×8 , intervalo GeLU	SAFE	4,24 s
Caso 5	FFN GPT-2 4×16 , intervalo GeLU	SAFE	20,10 s
9 provados · 3 com contraexemplo · 0 indecisos			

Fonte: `results/EVIDENCIAS.md`; logs brutos em `results/logs/`.

Três leituras equivocadas dessa tabela precisam ser bloqueadas de antemão. Um veredito SAFE vale para o *harness*, para as hipóteses declaradas e para o horizonte de desenrolamento indicado, e não para o sistema implantado. Um veredito UNSAFE é um resultado positivo do ponto de vista da verificação — o *solver* exibiu um traço concreto —, e não uma falha do experimento. E a ausência de linhas indecisas nesta execução específica não significa que a técnica sempre decida: várias configurações estudadas neste projeto **não** decidem, e estão reportadas nas subseções seguintes.

5.2 Caso 1 — redes quantizadas e *stubs* de inferência

O *harness* C quantizado que implementa XOR sobre as quatro entradas discretas foi provado em 0,08 s, e o *stub* em ponto flutuante da mesma topologia em 3,09 s. O *stub* de bloco MLP *gated*, no estilo DeepSeek, gera 28 condições de verificação — quatro remanescentes após simplificação — e é provado em 0,08 s.

Já o *harness* que impõe a propriedade de saída sobre a rede completa em ponto flutuante retorna UNSAFE em 197,82 s: existe entrada admissível para a qual a predição desquantizada excede o limite declarado. Esse resultado é informativo justamente por contrariar a expectativa induzida pela amostragem. A Figura 3 mostra a superfície de saída obtida por amostragem densa de 10^4 pontos, em que nenhuma violação aparece — um lembrete direto de que 10^4 amostras não cobrem um domínio contínuo, e de que a diferença entre “não encontrei” e “não existe” é exatamente o que a verificação formal entrega.

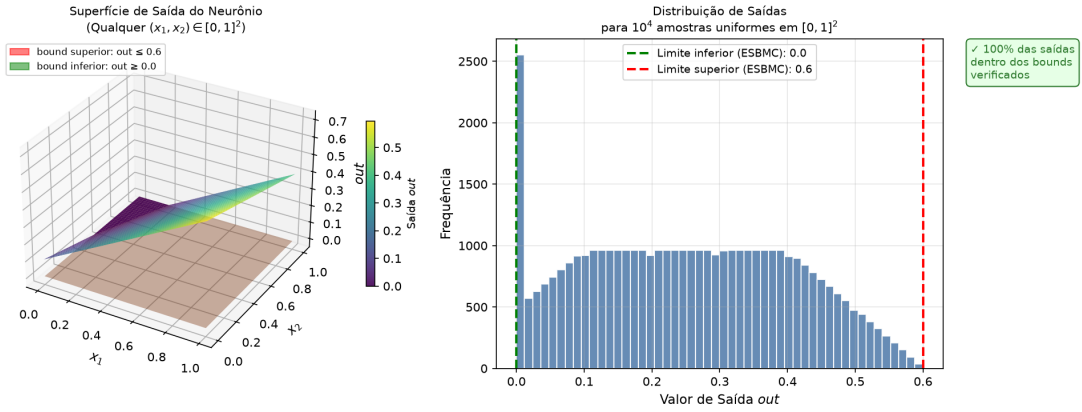


Figura 3: Superfície de saída do neurônio e distribuição amostral. A figura descreve a amostra e **não** constitui prova sobre o domínio contínuo.

Fonte: elaborada pelo autor.

Registre-se um limite de escopo desta frente. O plano previa executar a verificação diretamente sobre o modelo em Python, via `esbmc modelo.py -floatbv -k-induction`. O binário 6.8.0 versionado no repositório não possui o *frontend* Python compilado e rejeita arquivos `.py` com `failed to figure out type of file`; instalar as dependências de transpilação não contorna o problema, pois a limitação está no binário. Concluir esse experimento exige compilar a versão 8.0.0 com `-DENABLE_PYTHON_FRONTEND=ON`. Nenhum veredito é atribuído aqui ao modelo em Python.

5.3 Caso 2 — *kernels* de inferência e a barreira de escalabilidade

A verificação dos *kernels* de inferência produziu contraexemplo em 0,43 s, localizando violação de segurança de memória sob comprimento de sequência simbólico. Para o *kernel* GEMM com *tiling*, a Tabela 4 apresenta o estudo de escalabilidade.

Tabela 4: Escalabilidade da verificação do *kernel* GEMM em função da dimensão N .

Dimensão N	Tempo	Veredito
2	1,97 s	SAFE
3	16,83 s	SAFE
4	300,05 s	TIMEOUT (indeciso)

Fonte: `results/case2_ambiente.md`; Boolector, limite de 300 s, `-DDIM_LIMIT=N`.

De $N=2$ para $N=3$ o tempo cresce cerca de 8,5 vezes; em $N=4$ o *solver* não decide dentro do limite. A barreira observada, neste *harness* e nesta máquina, está em $N=4$. Três pontos — um deles indeciso — não sustentam extrapolação para dimensões de produção nem a proposição de uma lei de crescimento, e a conclusão permanece restrita ao experimento medido. A abordagem continua útil para verificar a corretude lógica do algoritmo de *tiling* em instâncias reduzidas, o que é coerente com a hipótese de escopo pequeno usualmente invocada nesse

contexto.

Cabe registrar como esse resultado foi obtido, porque o percurso é instrutivo. A versão anterior do experimento reportava uma curva de seis pontos, com tempos entre < 1 s e ≈ 15 s. O *script* de execução passava a dimensão por `-DDIM_LIMIT=<n>`, mas o *kernel* **nunca usava esse macro**: as dimensões estavam fixas em $M = N = K = 2$, e todos os “tamanhos” mediam o mesmo problema. Além disso, a *flag* `-smtlib` fazia o verificador *emitir* a fórmula em vez de resolvê-la. Corrigidos o macro, a extensão do arquivo e as *flags*, a curva suave desapareceu e a barreira em $N=4$ tornou-se visível.

5.4 Caso 3 — ciclo neuro-simbólico de geração e correção

O ciclo completou-se em **duas** iterações. Na iteração 0, o ESBMC detectou o *buffer overflow* do `strcpy` em 0,13 s de tempo de verificador, devolvendo o deslocamento exato da violação; na iteração 1, o código corrigido com `strncpy` recebeu VERIFICATION SUCCESSFUL em 0,11 s, encerrando o ciclo. As Figuras 4 e 5 apresentam a decomposição de tempo por iteração e a máquina de estados do protocolo.

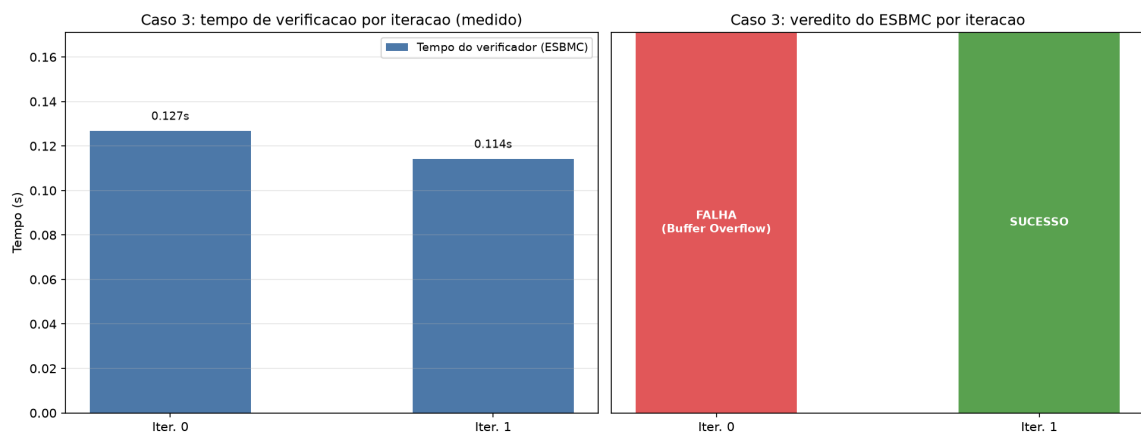


Figura 4: Tempo por iteração do ciclo gerar–verificar–corrigir, com decomposição entre atraso do gerador e tempo de verificação.

Fonte: elaborada pelo autor a partir de `results/case3_agent_stats.csv`.

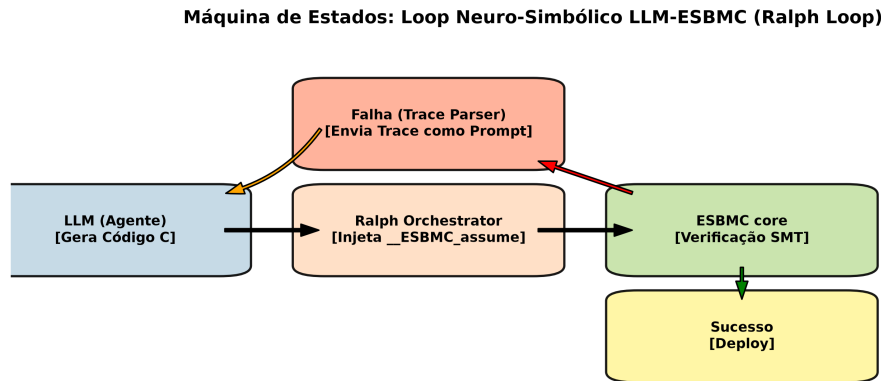


Figura 5: Máquina de estados do protocolo neuro-simbólico de geração e reparo.

Fonte: elaborada pelo autor.

O custo do verificador ficou abaixo de 0,15 s nas duas iterações, mas esse número só foi obtido após limitar o desenrolamento com `-unwind 32`: sem esse limite, o modelo de `strncpy` não termina; com `-no-unwinding-assertions`, o caso pode tornar-se vácuo. É um exemplo concreto de como um parâmetro aparentemente operacional altera o significado do veredito.

Duas correções foram necessárias para que este caso medisse alguma coisa. A versão anterior afirmava cinco iterações — inalcançáveis, já que o ciclo encerra na primeira prova bem-sucedida — e usava `-smtlib`, o que fazia com que o campo de sucesso fosse permanentemente falso e o arquivo de estatísticas ficasse com zero byte. O experimento reportado aqui usa o invólucro `core_verify` e produz o CSV de onde a Figura 4 é gerada.

Reafirme-se a limitação de escopo: o gerador é um *mock* determinístico. O experimento demonstra o protocolo e mede o custo do verificador dentro dele; conclusões sobre autonomia, qualidade de *prompts* ou desempenho de agentes reais exigem outro experimento.

5.5 Caso 4 — PID sob ruído não determinístico

O verificador provou, em **213,47 s**, que o controlador mantém $T < 150^{\circ}\text{C}$ ao longo dos dez passos e para *qualquer* ruído de sensor em $[-5, +5]$. A Figura 6 apresenta a dinâmica simulada sob cinco perfis distintos de ruído e a Figura 7 o retrato de fase correspondente.

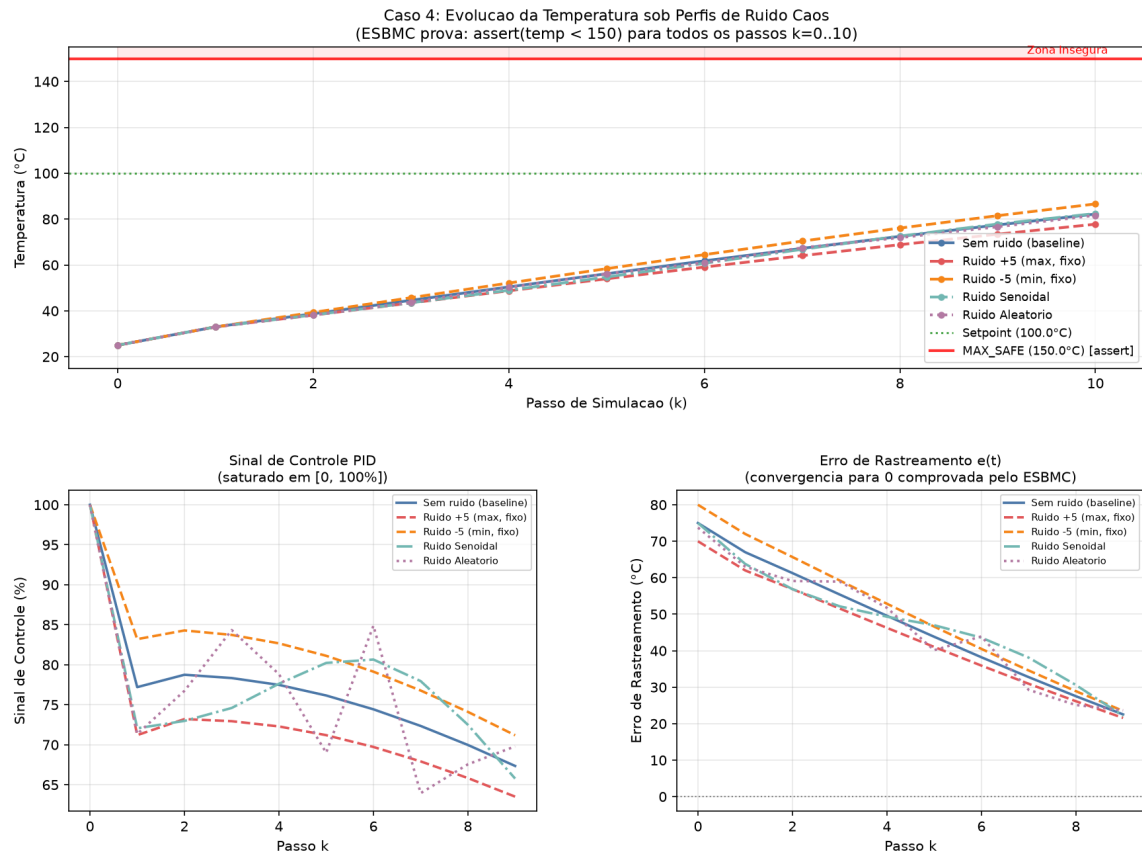


Figura 6: Evolução da temperatura, do sinal de controle e do erro de rastreamento sob cinco perfis simulados de ruído de sensor.

Fonte: elaborada pelo autor.

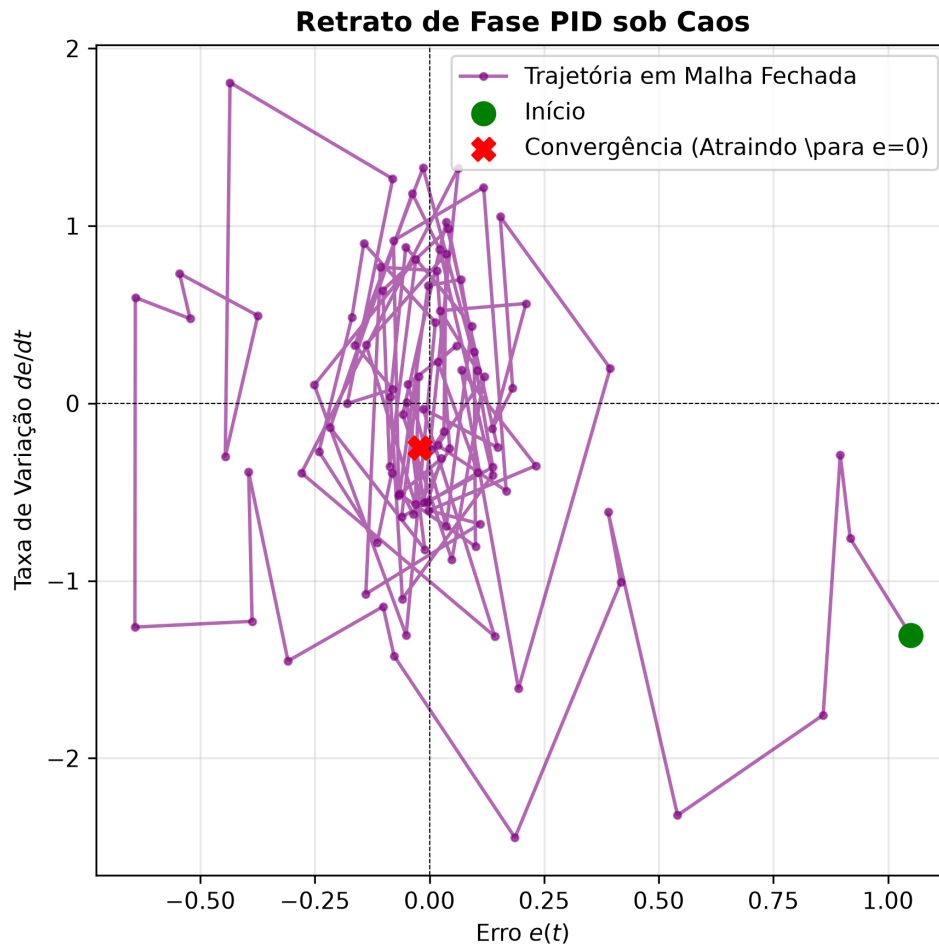


Figura 7: Retrato de fase do controlador PID sob perturbação.

Fonte: elaborada pelo autor, com semente fixa.

O erro de rastreamento diminui nos cinco perfis simulados, mas essa observação é finita e **não** comprova estabilidade assintótica. A prova formal cobre exatamente $T < 150^{\circ}\text{C}$, por dez passos, sob ruído de sensor em $[-5, +5]$; perturbações no atuador não foram modeladas.

Este caso produziu o episódio metodologicamente mais instrutivo do ciclo. Uma primeira medição, com limite de 200 s, resultou em *timeout*, e a afirmação esteve a ponto de ser classificada como não sustentada por evidência. Reexecutada com folga, a mesma propriedade foi **provada** — pouco além do limite anterior. A lição é simétrica e vale para todo o relatório: ler *timeout* como “seguro” inventa uma prova; lê-lo como “propriedade falsa” descarta uma prova legítima. Nos dois casos o defeito é o mesmo, tratar *ausência de sinal como sinal*. É por isso que a coluna de veredito do arquivo de evidências mantém TIMEOUT separado de SAFE e de UNSAFE, e nunca os colapsa.

5.5.1 Biblioteca de controle de terceiros

Para avaliar a aplicabilidade da abordagem a código de produção, a *Arduino-PID-Library* foi integrada como submódulo e submetida ao mesmo modelo de planta e de ruído. **A verificação não foi concluída.** Foram encontrados três obstáculos independentes, dois deles removidos:

1. o *harness* declarava `void __ESBMC_assume(int)`, em conflito com a declaração `bool` dos *intrinsics* do verificador, e por isso sequer era analisado — **corrigido**;
2. a macro `ARDUINO` não era definida, fazendo o código recair em um ramo anterior ao Arduino 1.0 que requer um cabeçalho inexistente — contornável por `-DARDUINO=100`;
3. **bloqueio remanescente**: o arquivo emprega **construtor delegante**, recurso de C++11, e o ESBMC 6.8.0 não oferece a opção `-std`, introduzida apenas na 8.0.0. O *harness* usa justamente o construtor de sete argumentos, que é o delegante, de modo que o recurso não pode ser evitado.

O registro relevante é que a barreira **não está** na complexidade do algoritmo de controle, mas na cobertura do *frontend* C++ do verificador e nas convenções de compilação do ecossistema alvo. O mesmo defeito de declaração do primeiro item foi encontrado e corrigido em outros dois arquivos do projeto, entre eles o *stub* de bloco MLP, que passou a ser verificável e figura como provado na Tabela 3.

5.6 Caso 5 — blocos *feed-forward* de modelos de linguagem

A Tabela 5 apresenta os tempos medidos do Método 2 — ativação abstraída por intervalos — sobre blocos FFN extraídos do GPT-2 e do LLaMA-7B.

Tabela 5: Verificação de blocos FFN com sobreaproximação intervalar da ativação (Método 2).

<i>Harness</i>	Dimensões	Ativação	Tempo	Veredito
<code>gpt2_2x4_qnn</code>	$2 \times 4 \times 2$	intervalo GeLU	0,24 s	SAFE
<code>llama-7b_4x8_qnn</code>	$4 \times 8 \times 4$	intervalo SiLU	3,46 s	SAFE
<code>gpt2_4x8_qnn</code>	$4 \times 8 \times 4$	intervalo GeLU	4,24 s	SAFE
<code>gpt2_4x16_qnn</code>	$4 \times 16 \times 4$	intervalo GeLU	20,10 s	SAFE

Fonte: `results/EVIDENCIAS.md`; ESBMC 6.8.0 com Boolector.

O crescimento entre 4×8 e 4×16 — de 4,24 s para 20,10 s ao dobrar a dimensão oculta — é coerente com a barreira observada no Caso 2 e delimita o alcance prático da técnica: a verificação é modular, aplicada a fatias representativas, e não ao modelo completo. Um bloco de produção com $d = 4096$ geraria da ordem de 16 milhões de multiplicações simbólicas.

O ponto conceitual mais importante deste caso é o compromisso entre as duas codificações. O Método 1, mais rápido, substitui a ativação por um *proxy* ReLU e portanto verifica um modelo *que não é o modelo implantado*. O Método 2 verifica a propriedade sobre uma relaxação intervalar, e a solidez do veredito para a ativação real depende inteiramente de os limites injetados serem conservadores. Nenhuma das duas codificações é “a correta”: a codificação mais rápida não é a mais sólida, e essa tensão precisa ser declarada junto do resultado.

Registre-se ainda que a documentação do subprojeto reporta um ganho de 18 a 27 vezes do Método 2 otimizado sobre uma codificação inicial que usava Z3, inteiros de 64 bits e mantinha

código morto de pré-ativação. Essa razão **não** foi remedida neste trabalho; apenas os tempos da Tabela 5 o foram, e ficaram sistematicamente abaixo dos documentados — 4,24 s contra 5,3 s, 3,46 s contra 4,3 s, 20,10 s contra 27 s —, diferença compatível com variação de máquina.

5.7 Caso 6 — política de aprendizado por reforço

O *solver* encontrou **contraexemplo** em 2,58 s: existe estado, dentro da caixa analisada, para o qual a ação da política ultrapassa o intervalo $[-1, 1]$ do atuador. A Figura 8 apresenta a avaliação da mesma expressão do *harness* sobre a mesma caixa de estados, na qual a violação é visível — a ação máxima atinge 1,50 contra o limite de 1,0, e uma fração não desprezível da grade viola o limite.

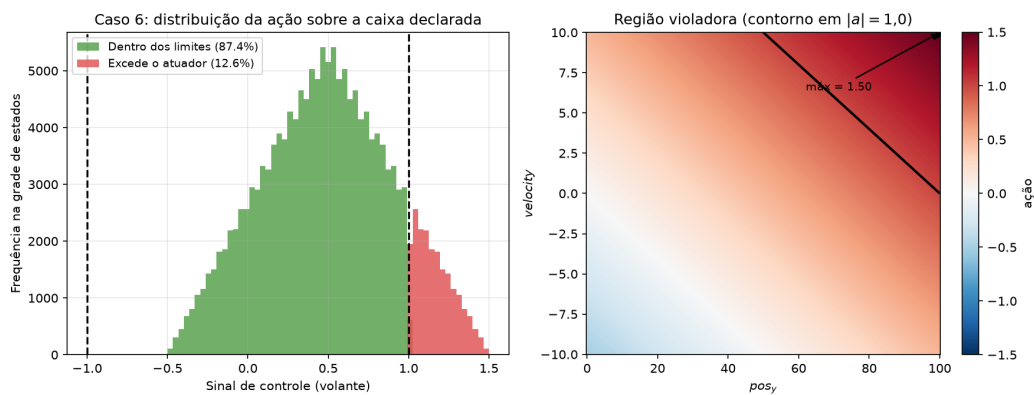


Figura 8: Distribuição das ações da política sobre a caixa de estados do *harness*: a região que excede o limite do atuador é atingível.

Fonte: elaborada pelo autor, avaliando a expressão do próprio *harness*.

A conclusão precisa ser enunciada com cuidado, porque é fácil invertê-la. O *harness* **detecta** a violação; ele **não** demonstra que o *safety shield* a impeça. O escudo precisa ser corrigido e novamente verificado antes de qualquer afirmação de proteção pré-implantação. Reportar esse resultado negativo é preferível a apresentá-lo como validação do escudo.

5.8 Caso 7 — malha fechada com controlador DDPG quantizado

5.8.1 Fidelidade da quantização

As três implementações da inferência — Python, TypeScript e C — coincidem bit a bit. A caracterização do erro de quantização sobre 10^4 amostras, com escala 256 e força em $[-10, +10]$ N, dá erro absoluto médio de 0,078 N e percentil 95 de 0,226 N, com erro absoluto máximo de 7,76 N. A cauda é relevante e está reportada: o erro típico é pequeno, mas o pior caso não é desprezível, e é precisamente o pior caso que a verificação formal explora.

5.8.2 Duas propriedades vácuas

O teste de vacuidade descrito na Seção 4.4 revelou que duas das três propriedades originalmente verificadas **não dependiam do controlador**:

- a propriedade de segurança em um passo asseria sobre $\theta_{\text{novo}} = \theta + 5\dot{\theta}/256$, expressão que não contém a força F . Executando a mesma asserção sem rede alguma e com F arbitrária, veredito e contraexemplo se reproduzem idênticos: os 745 pesos eram **código morto** para aquela propriedade;
- a propriedade de limite de força, $|F| \leq 10 \text{ N}$, vale por **construção** da aproximação de $\tanh$, que satura em Q8.8. Substituindo a rede inteira por uma entrada livre, a prova passa em 0,72 s.

A correção seguiu dois caminhos distintos. A primeira propriedade foi reescrita para **dois passos**, pois a força só alcança θ através de $\dot{\theta}$ no passo seguinte; a dependência foi confirmada observando que o contraexemplo muda conforme F é livre ou nula. A segunda foi **reclassificada** como verificação de sanidade da quantização — ela detecta erro de tabela ou de escala, o que é útil, mas não é prova sobre o ator — e uma propriedade nova foi introduzida para cobrir o que ela não podia decidir: na zona de perigo, o controlador deve aplicar força de magnitude mínima, condição que um controlador inerte viola e cuja decisão exige percorrer os pesos. A discriminação foi verificada nos dois sentidos: a nova propriedade falha para um controlador nulo e é provada para um controlador atuante.

Esta é a contribuição metodológica que o projeto considera mais transferível. Uma propriedade vácuca não produz erro, não produz aviso e não produz número implausível: produz uma prova verdadeira sobre o objeto errado. O único mecanismo que a expõe é o teste explícito de dependência.

5.8.3 Envelope de segurança e a barreira do BMC

A Tabela 6 caracteriza a propriedade corrigida de dois passos em função da velocidade angular inicial admitida.

Tabela 6: Segurança em dois passos em função do limite admitido de $|\dot{\theta}|$.

$ \dot{\theta} \leq (\text{Q8.8})$	rad/s	Veredito	Tempo
1280	5,00	UNSAFE	25,4 s
640	2,50	UNSAFE	28,8 s
320	1,25	Indeciso	> 300 s
160	0,62	Indeciso	> 300 s
80	0,31	Indeciso	> 300 s

Fonte: elaborada pelo autor; limite de 300 s por instância.

O resultado defensável é, portanto: a segurança de dois passos é **violada** acima de 2,5 rad/s e **indecisa** abaixo disso. **Não** se conclui que o controlador seja seguro em nenhuma faixa.

O comportamento da tabela contraria a intuição e merece explicação, porque **reduzir a região inicial torna o problema mais difícil**. Com uma caixa ampla existe contraexemplo, e encontrá-lo é a direção barata do problema: basta ao *solver* exibir uma testemunha. Nas caixas estreitas não há testemunha a exibir, e decidir exige *provar* a ausência de violação — a direção cara. A assimetria entre satisfatibilidade e insatisfatibilidade é, aqui, o fator dominante do custo.

A verificação de malha fechada obtida neste trabalho é de horizonte curto. Um *harness* de cinquenta passos chegou a ser esboçado e nunca foi concluído, e a leitura imediata seria falta de tempo. A medição mostra outra coisa: para a rede sintética, o custo de provar K passos cresce de 0,3 s e 46 MB em $K=1$ para **909 s e 383 MB** em $K=4$. A barreira não está em cinquenta passos; está em quatro. É limite do método, não do esforço.

5.8.4 Model checking indutivo como alternativa

A causa da barreira é estrutural: o BMC constrói *uma* fórmula contendo K cópias da relação de transição. O IC3/PDR nunca constrói essa fórmula, e seu resultado é um invariante indutivo — “seguro sempre” em vez de “seguro até K ”.

O sistema acoplado foi exportado como sistema de transição em Verilog e sintetizado pelo Yosys, resultando em **904.242 portas AND, 65 registradores e nível lógico 689**. A propriedade verificada é $|\theta| \leq 53$ em Q8.8, equivalente a aproximadamente 12° , a partir de uma caixa inicial de estados. A validação diferencial do Verilog gerado contra a referência coincidiu em 12 de 12 estados amostrados — o que detecta regressão de tradução, mas não prova equivalência universal. A Tabela 7 reúne os resultados.

Tabela 7: Bounded model checking contra model checking indutivo.

Método	Modelo	Veredito	Tempo	Memória
BMC (ESBMC)	sintético, $K=4$	seguro até 4 passos	909 s	383 MB
IC3 (ABC)	sintético com batente	provado $\forall t$	0,37 s	trivial
IC3 (ABC)	ator DDPG real	não convergiu	1802 s	708 MB
BMC (ABC)	ator DDPG real	5 frames, sem veredito	901 s	503 MB

Fonte: `ic3/EVIDENCIA.md`. Timeout é indeciso, não “seguro”.

No modelo sintético com batente mecânico, o IC3 produziu prova ilimitada em 0,37 s, com um invariante de quatro cláusulas sobre **três dos 65 bits de estado**: o algoritmo determinou por conta própria que os bits altos de θ permanecem iguais ao sinal e que as mais de cem mil portas da rede são irrelevantes para a prova. Nenhuma execução de BMC produz esse tipo de resultado, e a diferença de três ordens de grandeza no tempo não é a parte mais importante: a parte mais importante é que o veredito é *ilimitado*.

Sobre o controlador real, contudo, **nenhum dos dois métodos decide**. O PDR foi interrompido em 1802 s sem convergir e o BMC alcançou cinco *frames* em 901 s. É um empate técnico, e é assim que deve ser reportado — *timeout* é indeciso, e tratá-lo como “seguro” seria repetir exatamente o erro que este projeto passou o ciclo corrigindo.

Registre-se uma lacuna de evidência nesta medição: a saída bruta daquela execução de PDR **não foi preservada**, porque o *script* então filtrava a saída por palavras-chave antes de gravá-la. Restam tempo e pico de memória, medidos pelo invólucro. O defeito foi corrigido — a saída bruta passou a ser sempre salva —, mas a medição histórica permanece qualificada como tal em vez de ser apresentada como plenamente reproduzível.

A semelhança entre as duas barreiras — cinco *frames* no ator real de 745 parâmetros e $K=4$ na rede sintética, cerca de oito vezes menor — é sugestiva de um limite de escalabilidade, mas modelos, propriedades, representações e *solvers* diferem entre as duas medições. Elas motivam uma hipótese; não isolam causalidade nem demonstram que o limite pertença ao método.

A hipótese remanescente é de **representação**. A tradução para AIGER converte quatro palavras de estado em 65 bits isolados, dissolvendo exatamente a estrutura sobre a qual o PDR generaliza cláusulas: o invariante precisaria falar sobre θ *como palavra*, e não sobre bits soltos. Atacar essa hipótese exige exportar o sistema em BTOR2, que preserva as palavras, e empregar um motor de nível de palavra. Essa é a continuidade direta indicada na Seção 7, e é uma mudança de *representação*, não de motor.

5.9 Síntese comparativa

A Tabela 8 organiza os sete casos segundo o que efetivamente foi estabelecido e o que permanece em aberto.

Tabela 8: Síntese dos sete estudos de caso: o que foi estabelecido e sob quais condições.

Caso	O que foi estabelecido	Sob quais hipóteses	Em aberto
1	Rede quantizada provada; propriedade sobre a rede em ponto flutuante violada	Domínio discreto (XOR); intervalos por neurônio injetados	Verificação a partir do modelo em Python
2	Contraexemplo de memória; GEMM provado até $N=3$	$seq_len \in [1, 10]$; limite de 300 s	Dimensões de produção
3	Protocolo gerar-verificar-corrigir demonstrado em 2 iterações	Gerador <i>mock</i> determinístico; $-unwind\ 32$	Agente de linguagem real
4	$T < 150^{\circ}\text{C}$ provado por 10 passos	Ruído de sensor em $[-5, +5]$; sem perturbação de atuador	Estabilidade assintótica; biblioteca C++11
5	Quatro blocos FFN provados, até 4×16	Sobreaproximação intervalar da ativação	Solidez para a ativação real; escala $d=4096$
6	Contraexemplo: a ação excede o limite do atuador	Caixa de estados do <i>harness</i>	<i>Safety shield</i> corrigido e verificado
7	Vacuidades corrigidas; violação acima de 2,5 rad/s; barreira do BMC medida	Horizonte de dois passos; quantização Q8.8	Faixas baixas indecisas; representação word-level

Fonte: elaborada pelo autor.

Lida em conjunto, a tabela sugere uma regularidade que o projeto considera seu achado central. A verificação formal de componentes de IA é **praticável e informativa em escala modular**: blocos, *kernels* e redes pequenas são decididos em segundos, e os contraexemplos obtidos são acionáveis. O acoplamento com a planta, por outro lado, esbarra em uma barreira que não é de implementação nem de esforço, e que se manifesta em horizontes de poucos passos. Entre esses dois regimes está o espaço em que o método rende mais: propriedades locais, hipóteses declaradas explicitamente e vereditos qualificados pelo horizonte.

6 LIMITAÇÕES, AMEAÇAS À VALIDADE E DIFICULDADES ENCONTRADAS

6.1 Limitações do que foi provado

As garantias obtidas neste trabalho são condicionais, e a lista abaixo delimita exatamente a que condições:

- **Horizonte finito.** Todos os vereditos de BMC valem até o limite de desenrolamento indicado. A verificação em malha fechada cobre dois passos; o *harness* de cinquenta passos não foi concluído e a medição mostra que o custo torna-o inviável nesta máquina.

- **Indecisão sob faixas estreitas.** Abaixo de 1,25 rad/s, a propriedade de dois passos permanece indecisa em 300 s. Não há, portanto, faixa de operação em que o controlador esteja provado seguro.
- **Nenhum dos dois métodos decide o controlador real.** O PDR não convergiu em 1802 s e o BMC alcançou cinco *frames* em 901 s.
- **Sobreaproximação nas ativações.** Os quatro vereditos de blocos FFN valem para a relaxação intervalar injetada. Transferi-los para GeLU e SiLU reais exige que esses limites sejam conservadores, o que não foi provado — o *encoding* não representa a relação funcional exata.
- **Aproximação linear da sigmoide.** Onde a forma funcional foi mantida, a substituição por $\text{clip}(0,25z + 0,5, 0, 1)$ altera a função implantada; o veredito vale para a aproximação.
- **Perfis de caos com faixa contínua não decidem** sob `-floatbv`; apenas o perfil discreto de impulso decide, em 27 s.
- **Verificação a partir de Python não realizada.** O binário disponível não possui o *frontend* Python compilado.
- **Biblioteca de terceiros não verificada.** O bloqueio é a ausência de `-std` na versão 6.8.0, necessária para o construtor delegante de C++11.

6.2 Ameaças à validade

6.2.1 Validade de construção: a propriedade vácuua

A ameaça mais séria identificada no ciclo não é de desempenho, mas de significado. Uma propriedade que decide sem envolver o artefato produz uma prova verdadeira sobre um objeto que não interessa, e não emite nenhum sinal de erro. Duas das três propriedades da malha fechada estavam nessa condição, e foram descobertas apenas pelo teste explícito de dependência descrito na Seção 4.4. Não há razão para supor que o repositório esteja hoje livre de casos análogos; o que existe é um procedimento para detectá-los, e ele deve ser aplicado a toda propriedade nova.

6.2.2 Validade interna: fidelidade das traduções

Três traduções sustentam os resultados e nenhuma delas está provada correta. A quantização Q8.8 do ator coincide bit a bit entre três implementações, o que é evidência forte de consistência interna, mas não de fidelidade ao modelo treinado em ponto flutuante — a caracterização de erro mostra pior caso de 7,76 N. A tradução para Verilog foi validada por teste diferencial em doze estados: isso detecta regressão, não equivalência. E os *stubs* de blocos de *transformer* representam fatias reduzidas, não os módulos de produção.

6.2.3 Validade externa: escala e generalização

Os experimentos foram conduzidos em uma única máquina de quatro núcleos, com um binário específico do verificador. As barreiras medidas — $N=4$ no GEMM, $K=4$ na malha fechada, cinco *frames* no ABC — são propriedades daquele ambiente e daqueles *harnesses*, e não constantes do método. Extrapolá-las seria repetir o defeito que a auditoria do projeto encontrou e corrigiu.

6.2.4 Validade de conclusão: procedência dos números

Este relatório reporta apenas números com log bruto versionado, com uma exceção declarada: a execução histórica de PDR sobre o ator real, cuja saída bruta se perdeu antes da correção do *script*. Ela é apresentada como medição histórica do invólucro, e não como resultado plenamente reproduzível.

6.3 Dificuldades encontradas ao longo do ciclo

Registram-se as dificuldades que mais consumiram tempo, na expectativa de que sejam úteis a quem retomar o trabalho:

1. **Defeitos silenciosos na instrumentação.** Vários experimentos produziam números plausíveis sem medir nada: um parâmetro de dimensão que não chegava ao código compilado, uma *flag* que fazia o verificador emitir a fórmula em vez de resolvê-la, um portão de verificação comentado em um orquestrador. Nenhum deles emitia erro. Foram encontrados apenas por auditoria dirigida, e não por execução da suíte.
2. **Suíte de testes verde sem verificar.** A suíte chegou a passar sem exercitar o verificador, porque o resultado era reduzido a um booleano que também é falso quando nada executa. A reescrita do invólucro e a suíte de regressão de casos-limite foram respostas diretas a isso.
3. **Divergência entre binário e árvore de código.** O binário versionado é a versão 6.8.0 e a árvore presente no repositório é a 8.0.0, não compilada. *Flags* da 8.0.0 são rejeitadas pela 6.8.0 com código 64, facilmente confundível com propriedade violada. Compilar a 8.0.0 leva de uma a quatro horas, o que tornou a alternância custosa.
4. **Escolha de limites de tempo.** O episódio do PID mostra o custo de um limite arbitrário: 200 s produziram *timeout* para uma propriedade que é verdadeira e se prova em pouco mais que isso.
5. **Submódulo Git inconsistente.** Um *gitlink* sem entrada correspondente em `.gitmodules` fazia o *checkout* automatizado terminar com erro. O conteúdo foi recuperado — e não descartado — após identificar o repositório e o *commit* corretos.

6. **Peso do repositório.** Material de terceiros vendorizado responde por 308 MB dos 322 MB do diretório do projeto, o que penaliza clonagem e integração contínua. A decisão sobre o que preservar permanece em aberto.

7 CONCLUSÕES

Este relatório apresentou a aplicação de *bounded model checking* baseado em SMT, por meio do verificador ESBMC, a sete classes de artefatos de inteligência artificial, do neurônio quantizado isolado à malha fechada formada por planta *cart-pole* e controlador DDPG. Dos doze *harnesses* reexecutáveis consolidados, nove foram provados, três produziram contraexemplo e nenhum ficou indeciso, todos com saída bruta do *solver* versionada.

Quatro conclusões se sustentam sobre a evidência apresentada.

Primeira: a verificação formal de componentes de IA é praticável em escala modular e produz informação que a amostragem não produz. O caso mais eloquente é o do Caso 1, em que 10^4 amostras não exibem violação alguma e o *solver* encontra, em 197 s, uma entrada admissível que excede o limite declarado. Contraexemplos são acionáveis: no ciclo neuro-simbólico, o diagnóstico com deslocamento exato da violação fecha o ciclo de correção em uma iteração.

Segunda: a maior ameaça a este tipo de trabalho é a propriedade vácuca, e ela não emite sinal. Duas das três propriedades da malha fechada decidiam sem que os 745 parâmetros do controlador participassem da prova. Elas não produziam erro, aviso ou número implausível — produziam provas verdadeiras sobre o objeto errado. O único mecanismo que as expôs foi o teste explícito de dependência: remover o modelo e verificar se o veredito muda. Esse teste é barato e deveria ser rotina em qualquer verificação de rede neural.

Terceira: a barreira do BMC em malha fechada é do método, não do esforço, e foi medida. Provar K passos custa 0,3 s em $K=1$ e 909 s em $K=4$, com memória crescendo de 46 MB para 383 MB. O *harness* de cinquenta passos não ficou por fazer: ele é inviável nesta máquina. Reconhecer isso transforma a maior limitação do trabalho em resultado, desde que a comparação seja feita com honestidade.

Quarta: o *model checking* indutivo é a direção certa, mas a instância real exige atacar a representação, e não trocar de motor. No modelo sintético, o IC3 produziu prova *ilimitada* em 0,37 s, com invariante de quatro cláusulas sobre três dos 65 bits de estado, onde o BMC levou 909 s para quatro passos. Sobre o controlador real, entretanto, nenhum dos dois métodos decide: o PDR não convergiu em 1802 s e o BMC parou em cinco *frames* em 901 s. A hipótese mais plausível para esse impasse é a perda de estrutura na tradução: converter quatro palavras de estado em 65 bits isolados destrói exatamente aquilo sobre o que o PDR generaliza cláusulas.

Além dos vereditos, o ciclo entrega uma infraestrutura de reprodutibilidade que é, em si, um resultado: um invólucro que distingue erro de execução de propriedade violada, uma suíte

de regressão que congela essa distinção, um gerador que reexecuta cada afirmação do texto e versiona a saída bruta, e um índice que liga cada número deste relatório ao log que o produziu.

7.1 Perspectivas de continuidade

Cinco direções decorrem diretamente do que foi medido, em ordem de prioridade:

1. **Representação em nível de palavra.** Exportar o sistema de transição em BTOR2, que preserva as quatro palavras de estado em vez de dissolvê-las em bits, e empregar um motor word-level. É a continuidade direta do impasse do Caso 7 e a hipótese que o projeto considera mais promissora.
2. **Fechar a curva de escalabilidade do BMC** no modelo sintético, medindo $K=8$ e $K=16$, para caracterizar a barreira em vez de a supor. A medição deve permanecer no modelo sintético: no modelo real, cinco *frames* em 901 s já indicam que esses horizontes estão fora de alcance nesta máquina.
3. **Corrigir e verificar o *safety shield*** do Caso 6, de modo que o contraexemplo hoje obtido deixe de ser atingível, e reverificar a propriedade sobre o escudo corrigido.
4. **Compilar o ESBMC 8.0.0** com o *frontend* Python habilitado, o que desbloqueia simultaneamente a verificação a partir de modelos em Python e a da biblioteca de controle em C++11 — dois objetivos hoje impedidos pelo mesmo obstáculo.
5. **Substituir a sobreaproximação intervalar** das ativações por limites derivados de propagação de intervalos sobre os pesos reais, tornando o conservadorismo verificável em vez de assumido, e manter o teste de vacuidade das hipóteses como guarda permanente.

Encerra-se com a observação metodológica que atravessou todo o ciclo. O erro que mais frequentemente compromete trabalhos desta natureza não é provar algo falso — o *solver* raramente permite isso — mas **tratar ausência de sinal como sinal**: ler *timeout* como segurança, ler suíte verde como verificação, ler número plausível como medição. As correções mais importantes deste PIBIC foram todas dessa família, e a infraestrutura construída existe para que elas não se repitam silenciosamente.

8 CRONOGRAMA DE ATIVIDADES EXECUTADAS

A Tabela 9 apresenta a distribuição das atividades ao longo da vigência. As marcações correspondem aos períodos de execução; os bimestres anteriores a fevereiro de 2026 antecedem o registro versionado do projeto e devem ser conferidos contra o plano de trabalho aprovado antes da entrega **[preencher: conferir os dois primeiros bimestres com o plano aprovado]**.

Tabela 9: Cronograma executado, por bimestre da vigência 2025–2026.

Atividade	Bimestre					
	1º	2º	3º	4º	5º	6º
Revisão bibliográfica em verificação formal, BMC/SMT e verificação de redes neurais	X	X	X			
Preparação do ambiente e do repositório; estudo do ESBMC e de suas primitivas	X	X				
Caso 1 — redes quantizadas e <i>stubs</i> de inferência		X	X	X		
Caso 2 — <i>kernels</i> de inferência e estudo de escalabilidade			X	X		
Caso 3 — ciclo neuro-simbólico de geração e correção			X	X		
Caso 4 — controlador PID sob ruído e biblioteca de terceiros				X	X	
Caso 5 — blocos <i>feed-forward</i> de modelos de linguagem				X	X	
Caso 6 — política de aprendizado por reforço				X	X	
Caso 7 — malha fechada <i>cart-pole</i> com controlador DDPG					X	X
Construção da camada <code>core_verify</code> e da suíte de regressão					X	X
Auditoria de evidências, correção das propriedades vácuas e reexecução dos <i>harnesses</i>						X
Pipeline IC3/PDR: geração do sistema de transição, síntese e verificação indutiva						X
Redação do artigo, do relatório final e preparação da apresentação					X	X

Fonte: elaborada pelo autor.

8.1 Produtos gerados

A Tabela 10 relaciona os produtos entregues ao artefato correspondente no repositório, de modo que cada item seja verificável de forma independente.

Tabela 10: Produtos do ciclo e respectivos artefatos de verificação.

Produto	Artefato
Doze <i>harnesses</i> de verificação reexecutáveis	scripts/gerar_evidencias.py, results/logs/
Tabela consolidada de evidências	results/EVIDENCIAS.md
Invólucro programático com status triestado	core_verify/esbmc_caller.py
Analisador de contraexemplos	core_verify/SMT_feedback_parser.py
Suíte de regressão, incluindo os quatro modos de falha	tests/, com destaque para tests/test_ground-truth.py
Propriedades corrigidas da malha fechada e caracterização do envelope	cartpole/verify_ddpg_closed_loop.py, cartpole/caracteriza_prop_b.py
Pipeline de verificação ilimitada por IC3/PDR	ic3/gen_transition_system.py, ic3/validate_forward.py, ic3/run_pdr.sh
Evidência das medições indutivas	ic3/EVIDENCIA.md
Geradores e tabelas de consulta dos blocos FFN	cases/llm_ffn_verification/
Regeneração reproduzível das figuras	artigo/figs/gerar_figuras.sh
Documentação de uso e de limitações conhecidas	pibic/README.md, cartpole/ESBMC- NOTES.md
Backlog auditado com procedência de evidência por tarefa	KANBAN.md

Fonte: elaborada pelo autor.

8.2 Participação em eventos e produção associada

[preencher: listar aqui apresentação no Congresso de Iniciação Científica da UFAM, resumos submetidos, participação em eventos e eventuais publicações decorrentes do plano]

REFERÊNCIAS

- BARTO, A. G.; SUTTON, R. S.; ANDERSON, C. W. Neuronlike adaptive elements that can solve difficult learning control problems. **IEEE Transactions on Systems, Man, and Cybernetics**, SMC-13, n. 5, p. 834–846, 1983.
- BRADLEY, A. R. SAT-based model checking without unrolling. In: **Verification, Model Checking, and Abstract Interpretation (VMCAI)**. [S.l.: s.n.], 2011. p. 70–87.
- BRAYTON, R.; MISHCHENKO, A. ABC: An academic industrial-strength verification tool. In: **Computer Aided Verification (CAV)**. [S.l.: s.n.], 2010.
- CORDEIRO, L.; FISCHER, B.; MARQUES-SILVA, J. SMT-based bounded model checking for embedded ANSI-C software. **IEEE Transactions on Software Engineering**, v. 38, n. 4, p. 957–974, 2012.
- EÉN, N.; MISHCHENKO, A.; BRAYTON, R. Efficient implementation of property directed reachability. In: **Formal Methods in Computer-Aided Design (FMCAD)**. [S.l.: s.n.], 2011.
- GADELHA, M. R. *et al.* ESBMC 5.0: An industrial-strength C model checker. In: **Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering (ASE)**. [S.l.: s.n.], 2018. p. 888–891.
- HUANG, X. *et al.* Safety verification of deep neural networks. In: **Computer Aided Verification (CAV)**. [S.l.: s.n.], 2017. p. 3–29.
- KATZ, G. *et al.* Reluplex: An efficient SMT solver for verifying deep neural networks. In: **Computer Aided Verification (CAV)**. [S.l.: s.n.], 2017.
- KATZ, G. *et al.* The marabou framework for verification and analysis of deep neural networks. In: **Computer Aided Verification (CAV)**. [S.l.: s.n.], 2019.
- LILLICRAP, T. P. *et al.* Continuous control with deep reinforcement learning. In: **International Conference on Learning Representations (ICLR)**. [S.l.: s.n.], 2016.
- MOURA, L. de; BJØRNER, N. Z3: An efficient SMT solver. In: **Tools and Algorithms for the Construction and Analysis of Systems (TACAS)**. [S.l.: s.n.], 2008.
- NIEMETZ, A.; PREINER, M. Bitwuzla. In: **Computer Aided Verification (CAV)**. [S.l.: s.n.], 2023.
- NIEMETZ, A.; PREINER, M.; BIERE, A. Boolector 2.0 system description. **Journal on Satisfiability, Boolean Modeling and Computation**, 2014.
- SENA, L. *et al.* Verifying quantized neural networks using SMT-based model checking. In: **Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)**. [S.l.: s.n.], 2021. ArXiv:2106.05997.
- SONG, X. *et al.* QNNVerifier: A tool for verifying neural networks using SMT-based model checking. In: **International Conference on Software Engineering: Companion Proceedings (ICSE Companion)**. [S.l.: s.n.], 2022. ArXiv:2111.13110.
- WANG, S. *et al.* Efficient formal safety analysis of neural networks. In: **Advances in Neural Information Processing Systems (NeurIPS)**. [S.l.: s.n.], 2018.

WOLF, C. **Yosys Open SYnthesis Suite**. 2016. Disponível em: <<https://yosyshq.net/yosys/>>.

APÊNDICE A — REPRODUTIBILIDADE

A.1 Preparação do ambiente

O binário do verificador acompanha o repositório, de modo que não há etapa obrigatória de compilação para reproduzir os resultados reportados na Seção 5.

```
git clone --recurse-submodules <url-do-repositorio>
cd esbmc/pibic
python3 -m venv .venv && source .venv/bin/activate
python -m pip install -e "[test]"
pytest tests/ -v
```

O invólucro localiza o binário nesta ordem: a variável `$ESBMC_BIN`; `esbmc` no `$PATH`; uma compilação local em `../build/src/esbmc/esbmc`; e, por fim, o binário versionado. Se nenhum existir, a suíte é **pulada** com motivo explícito, em vez de acumular erros que seriam confundidos com falhas de propriedade.

A.2 Reexecução das evidências

```
python scripts/gerar_evidencias.py          # reexecuta os 12 harnesses
# saida bruta -> results/logs/*.log
# tabela      -> results/EVIDENCIAS.md
```

A.3 Verificação ilimitada por IC3/PDR

```
sudo apt-get install -y yosys              # traz o ABC embutido
cd ic3
python3 gen_transition_system.py --bits 16 -o cl_ddpg16.v
python3 validate_forward.py                # teste diferencial em 12 estados
./run_pdr.sh cl_ddpg16.v 1800
```

A execução de `validate_forward.py` não é opcional: ela compara o Verilog gerado com a referência em ponto fixo antes de qualquer medição. A coincidência das doze amostras detecta regressão de tradução; não constitui prova de equivalência universal.

A.4 Compilação deste relatório

```
cd pibic/relatorio_final
make          # pdflatex -> bibtex -> pdflatex -> pdflatex
make clean    # remove os intermediarios
```

As figuras são localizadas por `\graphicspath` em `figuras/` e, subsidiariamente, em `../artigo/figs/`. São necessários os pacotes `texlive-latex-base`, `texlive-latex-recommended`, `texlive-latex-extra`, `texlive-fonts-recommended`,

`texlive-lang-portuguese` e `texlive-publishers`, este último por fornecer `abntex2cite` e o estilo bibliográfico `abntex2-alf`.

A.5 Advertências operacionais

- **Nunca testar `not r.is_safe` para concluir que um defeito foi encontrado.** Esse valor também é falso quando o verificador não executou. Use `r.status == UNSAFE`.
- **`-bounds-check` e `-pointer-check` não existem** no ESBMC: essas verificações são ativadas por padrão e a ferramenta expõe apenas as formas negativas, `-no-bounds-check` e `-no-pointer-check`. Passar as formas inexistentes encerra o processo com código 64.
- **TIMEOUT é indeciso**, nunca “seguro”.
- **`-no-unwinding-assertions` só é seguro em *harnesses* sem laços.** Em código com laços, o uso da opção pode tornar o caso vácuo.

APÊNDICE B — ÍNDICE DE EVIDÊNCIAS

A Tabela 11 liga cada veredito citado neste relatório ao comando que o produziu e ao arquivo de saída bruta correspondente, todos sob `results/logs/`.

Tabela 11: Veredito, comando e log bruto de cada *harness*.

<i>Harness</i>	Opções de verificação	Veredito	Log
MLP quantizada (XOR)	-no-unwinding-assertions	SAFE	verify_mlp_-qnn.log
Stub da MLP	-floatbv -no-unwinding-assertions	SAFE	mlp_stub.log
Rede + propriedade	-floatbv -no-unwinding-assertions	UNSAFE	property_-check.log
Stub MLP <i>gated</i>	-floatbv -overflow-check	SAFE	deepseek_mlp_-stub.log
Kernels de inferência	-memory-leak-check -overflow-check -no-pointer-check -no-unwinding-assertions -z3 -unwind 4	UNSAFE	kernels_-benchmarks.log
GEMM <i>tiling</i> , $N=3$	-memory-leak-check -overflow-check -no-unwinding-assertions -boolector -unwind 10 -DDIM_LIMIT=3	SAFE	matmul_kernel.log
PID, 10 passos	-floatbv -no-unwinding-assertions -unwind 11	SAFE	pid_-controller.log
Política de RL	-floatbv -z3 -unwind 1	UNSAFE	rl_policy.log
FFN GPT-2 2×4	-boolector -no-unwinding-assertions	SAFE	gpt2_2x4_qnn.log
FFN GPT-2 4×8	-boolector -no-unwinding-assertions	SAFE	gpt2_4x8_qnn.log
FFN LLaMA-7B 4×8	-boolector -no-unwinding-assertions	SAFE	llama-7b_4x8_-qnn.log
FFN GPT-2 4×16	-boolector -no-unwinding-assertions	SAFE	gpt2_4x16_qnn.log

Fonte: `results/EVIDENCIAS.md`. Duração total da execução consolidada: 463 s.

B.1 Medições complementares

As medições abaixo não integram a execução consolidada e possuem procedência própria, declarada em cada caso.

Tabela 12: Medições complementares e sua procedência.

Medição	Resultado	Procedência
Escalabilidade do GEMM, $N \in \{2, 3, 4\}$	1,97 s / 16,83 s / TIMEOUT	<code>results/case2_ambiente.md</code>
Ciclo neuro-simbólico, 2 iterações	0,13 s (UNSAFE) e 0,11 s (SAFE)	<code>results/case3_agent_-stats.csv</code>
Envelope de segurança em dois passos	violado acima de 2,5 rad/s; indeciso abaixo	execução direta, limite de 300 s
BMC na rede sintética, $K=1$ a $K=4$	0,3 s/46 MB a 909 s/383 MB	execução direta
IC3 na rede sintética com batente	provado $\forall t$ em 0,37 s	<code>ic3/EVIDENCIA.md</code>
BMC do ABC sobre o ator real	5 frames, 901,4 s, 503 MB	<code>ic3/cl_ddpg16.bmc.out</code>
PDR do ABC sobre o ator real	não convergiu, 1802,4 s, 708 MB	medição histórica do invólucro; saída bruta não preservada
Síntese do sistema de transição	904.242 portas AND, 65 registradores, nível 689	<code>ic3/EVIDENCIA.md</code>
Erro de quantização Q8.8 do ator	médio 0,078 N; p95 0,226 N; máximo 7,76 N	<code>cartpole/quantization_-report.json</code>

Fonte: elaborada pelo autor.